



MANIPAL INSTITUTE OF TECHNOLOGY

MANIPAL

(A constituent unit of MAHE, Manipal)

Department of Electrical & Electronics Engineering

Digital System Design Lab – III Semester

MINI PROJECT REPORT

TITLE: Design of a Car Parking Management System and implementation
on Nexys DDR4

Submitted by:

Name	Roll Number	Registration Number	Batch
Kritagya Tripathi	23	240906504	C1
Atla Durga Harsha Vardhan Reddy	03	240906010	C1
Hrishikesh Dhabade	04	240906022	C1
Aravindan SP	08	240906104	C1

Department of Electrical and Electronics Engineering

Manipal Institute of Technology, Manipal

Aug- Dec 2025

	CONTENTS	Page No.
1.	Abstract /Description about the work	
2.	Background theory and details about NEXYS4 DDR FPGA	
3.	Methodology with Block diagram or Flow chart	
4.	Design/state diagram/truth table	
5.	Simulation and Implementation Results, Result analysis	
6.	Contribution of each student	
7.	References	

1. Abstract /Description about the work

Efficient parking management is a growing challenge in urban areas due to increasing vehicles and limited spaces. Traditional manual systems often lead to errors and congestion. This project introduces an automated Parking Management System using FPGA technology to track vehicles and display the number of Parking Slots available.

The system employs digital counters, combinational logic, and finite state machines to monitor vehicles entering and exiting. IR sensors, simulated by FPGA switches, detect vehicles, while a seven-segment display shows available slots. LED indicators signal parking availability or full status, and a servo-controlled gate, driven by PWM, manages vehicle entry.

Implemented in Verilog HDL and tested with Vivado, the design runs on the Nexys4 DDR FPGA board. This project delivers a reliable, cost-effective, and scalable real-time parking solution that improves convenience and efficiency.

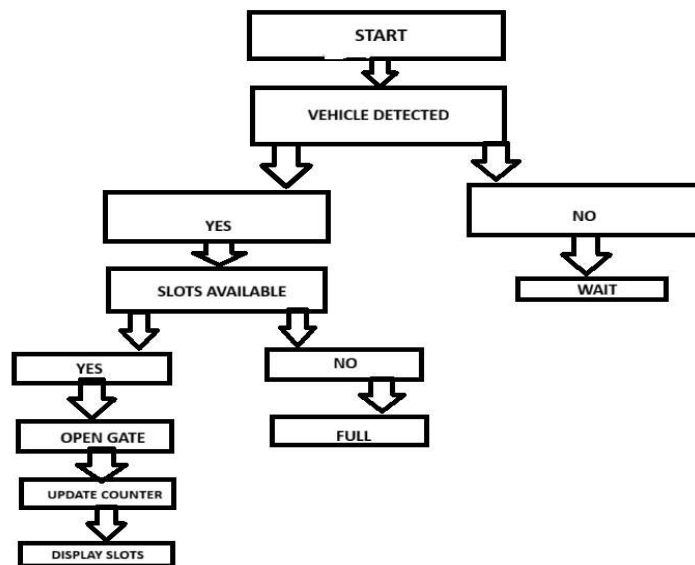
2. Background theory and details about NEXYS4 DDR FPGA

- Nexys4 DDR FPGA board features the Xilinx Artix-7 chip with built-in switches, buttons, LEDs, seven-segment displays, and communication interfaces.
- Onboard switches are used for inputs like password entry and sensor simulation.
- LEDs indicate system status clearly.
- Seven-segment display shows available parking slots in real-time.
- PWM signals generated in Verilog control a servo motor for gate movement.
- Vivado Design Suite handles simulation, synthesis, and FPGA programming.
- This hardware-software setup enables an efficient and reliable parking management system

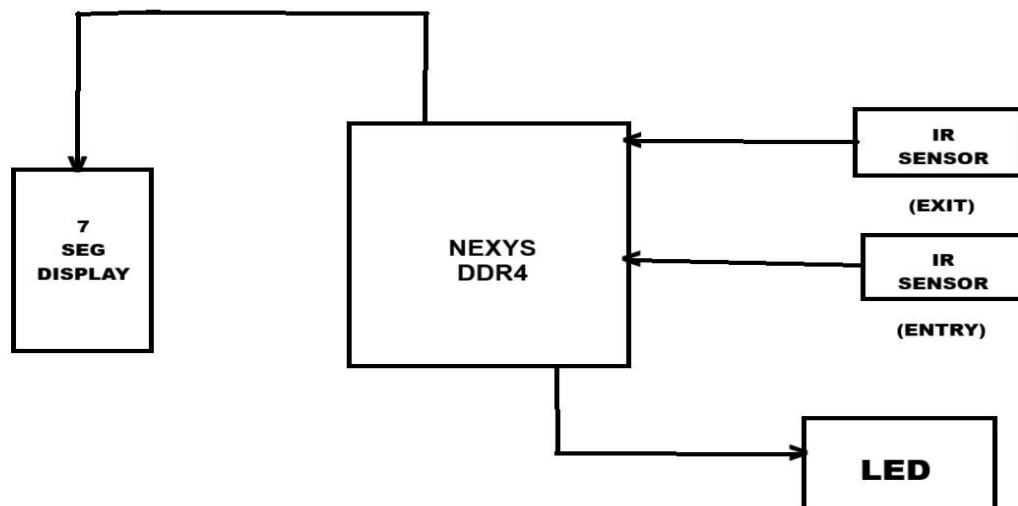
3. Methodology with Block diagram or Flow chart

- Designed a digital vehicle counter that increases when a vehicle enters and decreases when one leaves.
- Created logic to show parking status: LEDs indicate if slots are available or full, and a seven-segment display shows the number of free spots.

- Developed a password-based finite state machine (FSM) to ensure only authorized access can open the gate.
- Implemented servo motor control using PWM signals to smoothly open and close the gate as needed.
- Built sensor interfaces with edge detection and debouncing to prevent false readings or glitches.
- We Wrote the entire design in Verilog, simulated and tested it in Vivado, and assigned the signals to the FPGA pins for real-world implementation.



4. Design/state diagram/truth table



Truth Table:

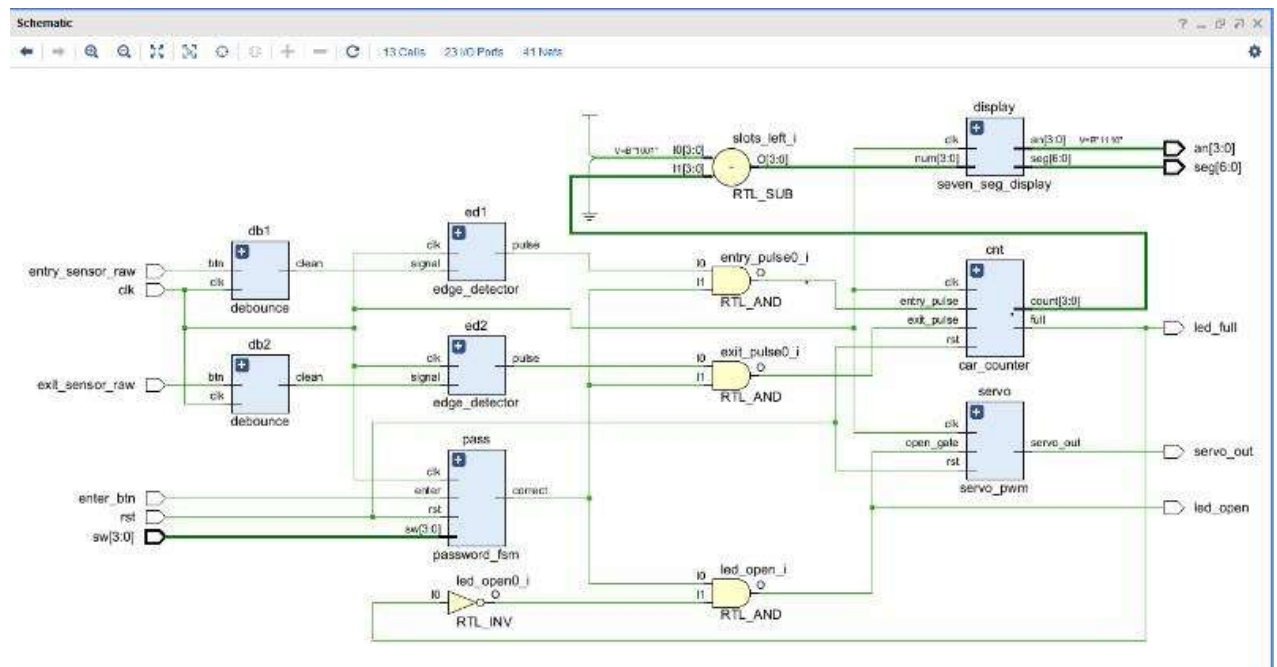
Entry	Exit	Current Count	Next Count
1	0	N	N+1
0	1	N	N-1
0	0	N	N
1	1	N	N

Pin Configuration:

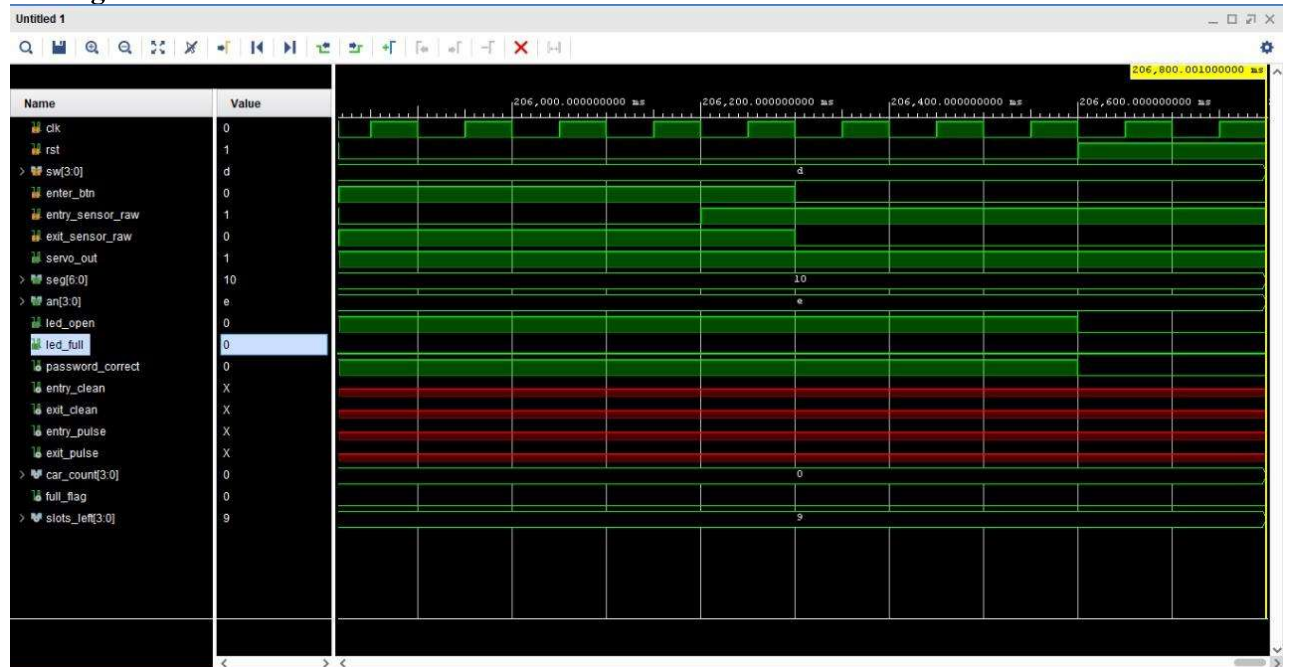
Name	Direction	Package Pin	I/O std
an(4)			
an[3]	OUT	J14	LVC MOS33
an[2]	OUT	T9	LVC MOS33
an[1]	OUT	J18	LVC MOS33
an[0]	OUT	J17	LVC MOS33
seg(7)			
seg[6]	OUT	L18	LVC MOS33
seg[5]	OUT	T11	LVC MOS33
seg[4]	OUT	P15	LVC MOS33
seg[3]	OUT	K13	LVC MOS33
seg[2]	OUT	K16	LVC MOS33
seg[1]	OUT	R10	LVC MOS33
seg[0]	OUT	T10	LVC MOS33
sw(4)			
sw[3]	IN	J15	LVC MOS33
sw[2]	IN	L16	LVC MOS33
sw[1]	IN	M13	LVC MOS33
sw[0]	IN	R15	LVC MOS33
Scalar Ports			
clk	IN	E3	LVC MOS33
enter_btn	IN	P18	LVC MOS33
entry_sensor_raw	IN	A13	LVC MOS33
exit_sensor_raw	IN	A14	LVC MOS33
led_full	OUT	V11	LVC MOS33
led_open	OUT	V12	LVC MOS33
rst	IN	N17	LVC MOS33
servo_out	OUT	T13	LVC MOS33

5. Simulation and Implementation Results, Result analysis

- Simulation waveforms verified functionality of counter, FSM, and PWM.
- Gate opens only if password correct and slots available.
- LEDs and seven-segment correctly indicate status.



Verilog Code



1. Top Module:

```
module top_parking(
    input wire clk,
    input wire rst,
    input wire [3:0] sw,           // password switches
    input wire enter_btn,         // enter button
    input wire entry_sensor_raw,  // entry IR
    input wire exit_sensor_raw,   // exit IR
    output wire servo_out,        // servo output PWM
    output wire [6:0] seg,        // 7-segment segments
    output wire [3:0] an,         // 7-segment enable (anodes)
    output wire led_open,        // gate open LED
    output wire led_full         // parking full LED
);

    // internal signals
    wire password_correct;
    wire entry_clean, exit_clean;
    wire entry_pulse, exit_pulse;
    wire [3:0] car_count;
    wire full_flag;
    wire [3:0] slots_left;
    // fixed number of total slots = 9
    assign slots_left = 9 - car_count;

    // --- PASSWORD CHECK ---
    password_fsm pass(
        .clk(clk),
        .rst(rst),
        .sw(sw),
        .enter(enter_btn),
        .correct(password_correct)
    );
endmodule
```

```

// --- SENSOR CLEANUP ---
debounce db1(.clk(clk), .btn(entry_sensor_raw), .clean(entry_clean));
debounce db2(.clk(clk), .btn(exit_sensor_raw), .clean(exit_clean));
// --- EDGE DETECTION ---
edge_detector ed1(.clk(clk), .signal(entry_clean), .pulse(entry_pulse));
edge_detector ed2(.clk(clk), .signal(exit_clean), .pulse(exit_pulse));
// --- CAR COUNT (only if password correct) ---
car_counter cnt(
    .clk(clk),
    .rst(rst),
    .entry_pulse(entry_pulse & password_correct), // count only if gate open
    .exit_pulse(exit_pulse & password_correct),
    .count(car_count),
    .full(full_flag)
);
// --- SERVO CONTROL ---
servo_pwm servo(
    .clk(clk),
    .rst(rst),
    .open_gate(password_correct && !full_flag),
    .servo_out(servo_out)
);

// --- SEVEN SEGMENT DISPLAY ---
seven_seg_display display(
    .clk(clk),
    .num(slots_left),
    .seg(seg),
    .an(an)
);
// --- LED INDICATORS ---
assign led_open = password_correct && !full_flag;
assign led_full = full_flag;
endmodule

```

2. Seven Segment Display Driver

```

module seven_seg_display(
    input wire clk,
    input wire [3:0] num,
    output reg [6:0] seg,
    output reg [3:0] an
);
    always @(*) begin
        case(num)
            4'd0: seg = 7'b1000000;
            4'd1: seg = 7'b1111001;
            4'd2: seg = 7'b0100100;
            4'd3: seg = 7'b0110000;
            4'd4: seg = 7'b0011001;
            4'd5: seg = 7'b0010010;
            4'd6: seg = 7'b0000010;
            4'd7: seg = 7'b1111000;
            4'd8: seg = 7'b0000000;
            4'd9: seg = 7'b0010000;
            default: seg = 7'b1111111;
        endcase
        an = 4'b1110; // enable only rightmost digit
    end
end

```



```
endmodule
```

3. Servo PWM

```
module servo_pwm(  
    input wire clk,  
    input wire rst,  
    input wire open_gate,  
    output reg servo_out  
);  
    parameter CLK_FREQ = 100_000_000; // 100 MHz  
    parameter PERIOD = CLK_FREQ / 50; // 20 ms period  
    parameter MIN_PULSE = 1_000_000; // 1 ms  
    parameter MAX_PULSE = 2_000_000; // 2 ms  
  
    reg [31:0] counter = 0;  
    reg [31:0] pulse_width = MIN_PULSE;  
  
    always @(posedge clk or posedge rst) begin  
        if (rst)  
            counter <= 0;  
        else if (counter >= PERIOD - 1)  
            counter <= 0;  
        else  
            counter <= counter + 1;  
    end  
  
    always @(posedge clk or posedge rst) begin  
        if (rst)  
            pulse_width <= MIN_PULSE;  
        else if (open_gate)  
            pulse_width <= MAX_PULSE;  
        else  
            pulse_width <= MIN_PULSE;  
    end  
    end  
    always @(*) begin  
        servo_out = (counter < pulse_width);  
    end  
endmodule
```

4. Car Counter

```
module car_counter(  
    input wire clk,  
    input wire rst,  
    input wire entry_pulse,  
    input wire exit_pulse,  
    output reg [3:0] count,  
    output reg full  
);  
    parameter MAX_SLOTS = 9;  
    always @(posedge clk or posedge rst) begin  
        if (rst) begin  
            count <= 0;  
            full <= 0;  
        end else begin  
            if (entry_pulse && count < MAX_SLOTS)
```

```

        count <= count + 1;
    else if (exit_pulse && count > 0)
        count <= count - 1;
    full <= (count >= MAX_SLOTS);
end
end
endmodule

```

5. Edge Detector

```

module edge_detector(
    input wire clk,
    input wire signal,
    output reg pulse
);
    reg prev = 0;
    always @(posedge clk) begin
        pulse <= (~prev) & signal;
        prev <= signal;
    end
endmodule

```

6. Debounce Module

```

module debounce(
    input wire clk,
    input wire btn,
    output reg clean
);
    reg [19:0] counter = 0;
    reg btn_sync = 0;
    always @(posedge clk) begin
        btn_sync <= btn;
        if (btn_sync != clean)
            counter <= counter + 1;
        else
            counter <= 0;
        if (counter == 1_000_000)
            clean <= btn_sync;
    end
endmodule

```

7. Password FSM

```

module password_fsm(
    input wire clk,
    input wire rst,
    input wire [3:0] sw,
    input wire enter,
    output reg correct
);
    parameter PASSWORD = 4'b1101;

    always @(posedge clk or posedge rst) begin
        if (rst)
            correct <= 0;
        else if (enter)
            correct <= (sw == PASSWORD);
    end
end

```

endmodule

6. Result Analysis

- The system effectively prevents vehicles from entering when the parking is full, avoiding overcrowding.
- Password logic reliably restricts access to authorized owner only.
- The servo-controlled gate and LED indicators provide clear, real-time feedback on parking status.
- The project successfully demonstrates practical applications of counters, finite state machines (FSMs), and combinational logic on an FPGA platform.

7. Contribution of Each Student

Kritagya Tripathi – Top Parking (Code) and Car Counter (Code)

Harsha Vardhan Reddy – Debounce (Code) and Password (Code)

Hrishikesh Dhabade – Documentation and Servo Pwm (Code)

Aravindan SP – Seven Segment Display (Code) and Edge Trigger (Code)

8. REFERENCES

Nexys DDR4 Reference Manual.

Digital System and Computer Architecture – David Harris.

Xilinx Vivado Design Suite Documentation.

IEEE papers on FPGA-based traffic and parking management systems.